



ENGENHARIA DA COMPUTAÇÃO
UNIVERSIDADE FEDERAL DO MARANHÃO



NEURAL TURING MACHINE

Docente: Dr. Thales Levi Azevedo Valente

Discentes: Alisson Diniz, Caio Reis e Luís Guilherme





SUMÁRIO

Introdução - 3

Maquina de Turing - 4

Fundamentação Teórica - 5

Comparações & Limitações - 11

Futuro & Desafios Abertos - 12

Nosso Projeto - 13

Referências - 31

Agradecimentos - 32

Por que Neural Turing Machines?

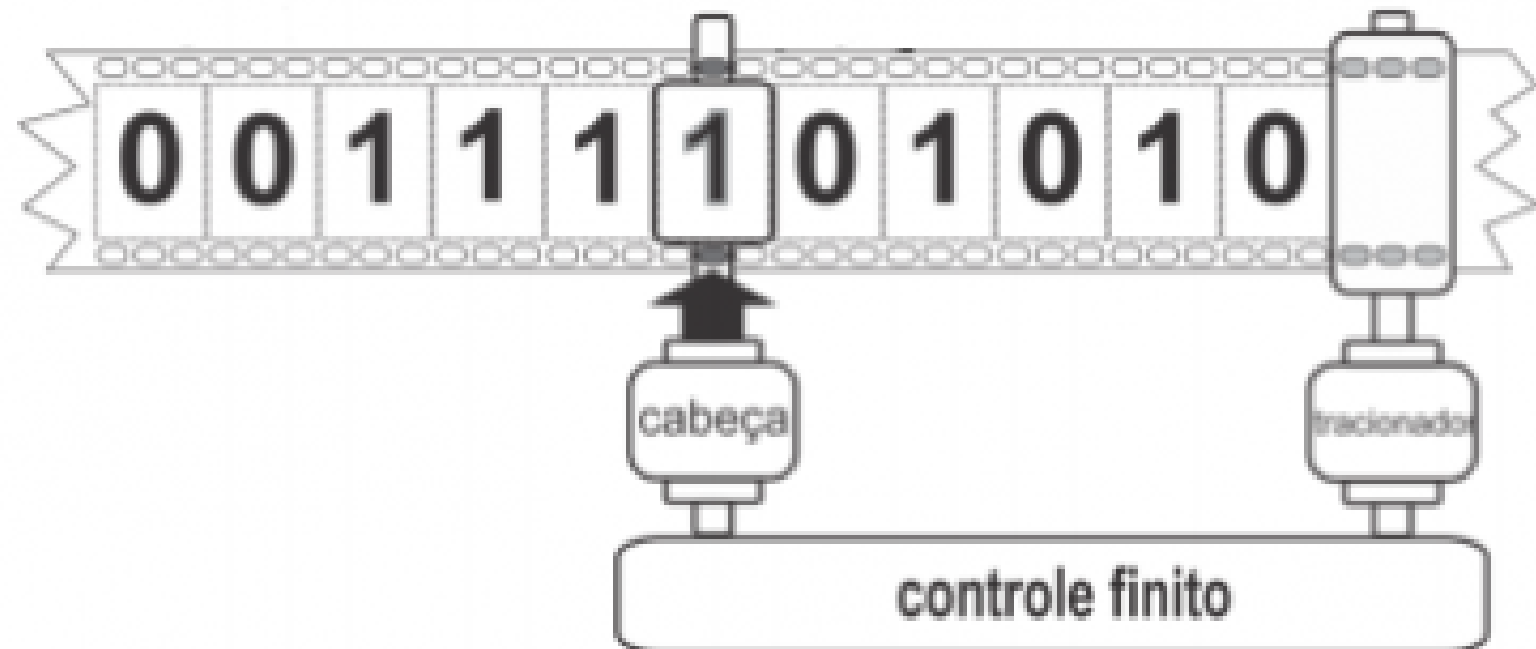
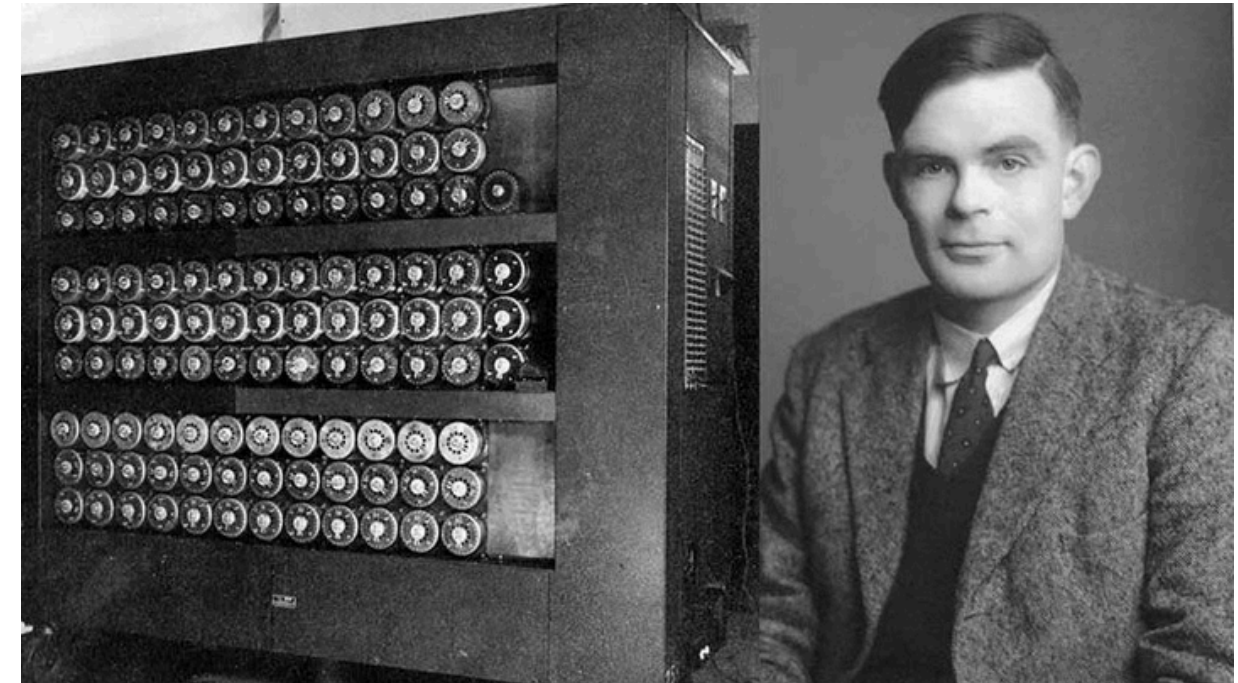
- **Desafio na IA:**
 - Aprender operações sequenciais complexas
 - Manipular memória e estruturas de dados
 - Lidar com dependências de longo prazo
- **Limitações das redes clássicas:**
 - RNNs e LSTMs não manipulam memória de forma explícita
 - Dificuldade em aprender comportamentos algorítmicos



MÁQUINA DE TURING

Máquina de Turing – Base da Computação

- **Definição (em tópicos):**
 - Modelo teórico de computação universal
 - Composta por:
 - Fita infinita (memória)
 - Cabeça de leitura/escrita
 - Conjunto finito de estados
 - Função de transição
- **Função:**
 - Descreve algoritmos passo a passo
 - Base para o conceito de computabilidade



FUNDAMENTAÇÃO TEÓRICA

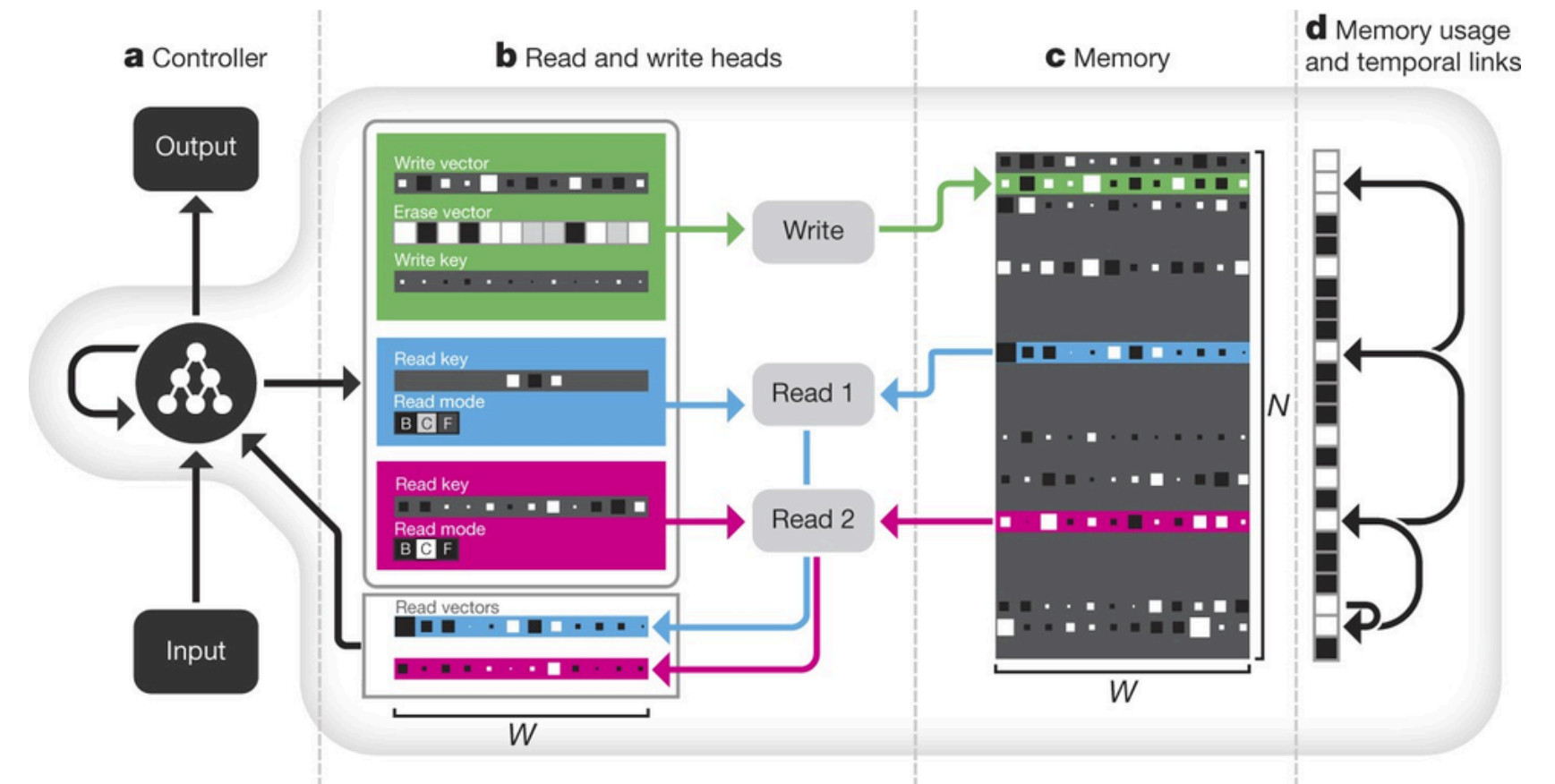
O que é a Neural Turing Machine (NTM)?

- **Conceito:**

- Proposta em 2014 por Alex Graves (DeepMind)
- Combina:
 - Redes recorrentes (RNNs)
 - Memória externa diferenciável

- **Objetivo:**

- Superar limitações de RNNs/LSTMs
- Aprender algoritmos, lidar com dados ordenados
- Armazenar e manipular memória de longo prazo



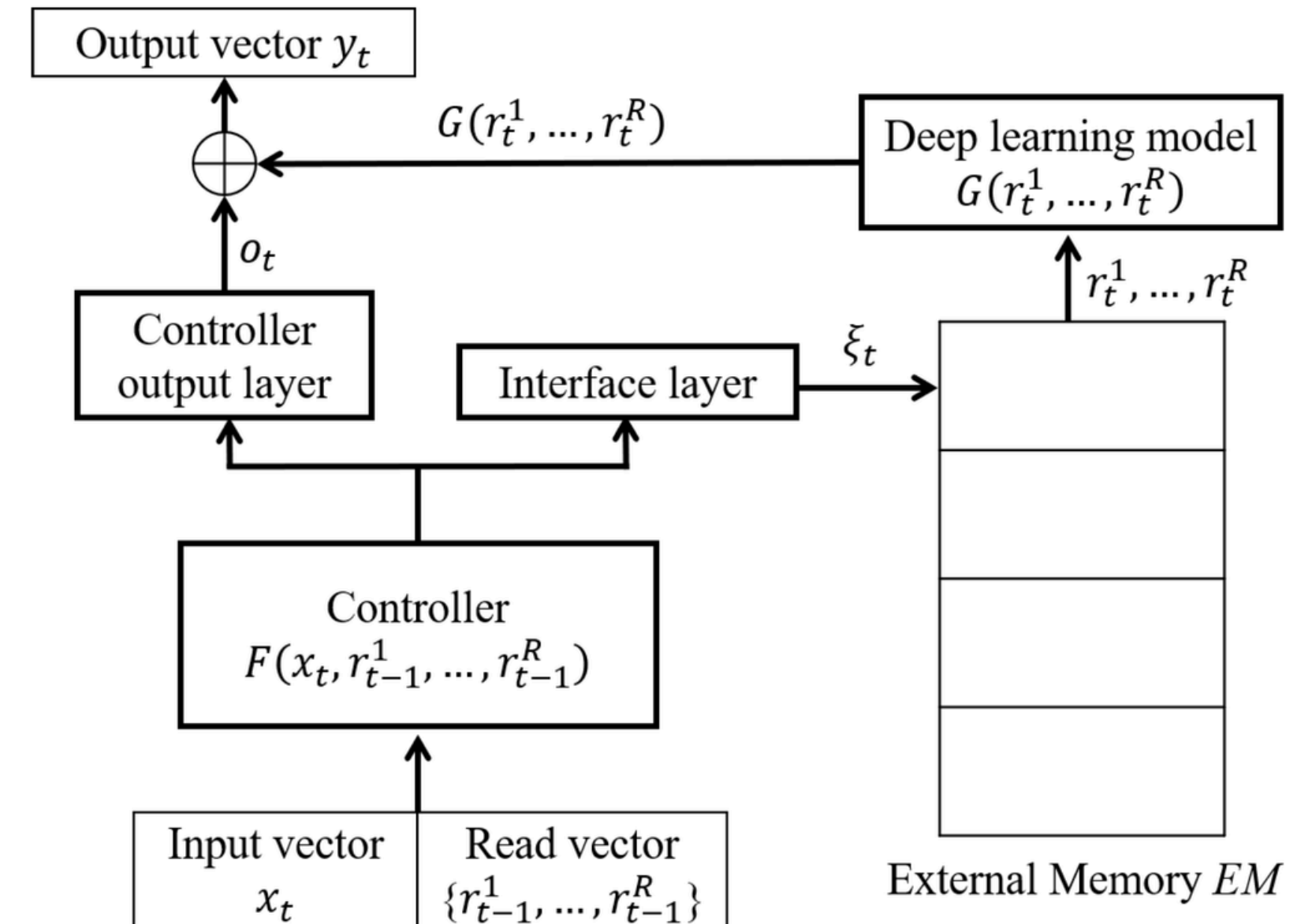
EXEMPLO

Exemplo: Funcionamento da NTM passo a passo

Fluxo geral:

1. Recebe um vetor de entrada x_t
2. Controller processa com base no input e no que leu antes da memória
3. Gera uma interface de leitura/escrita (ξ_t) para acessar a memória
4. A memória gera um vetor de leitura r_t
5. O controller usa isso para produzir a saída final y_t

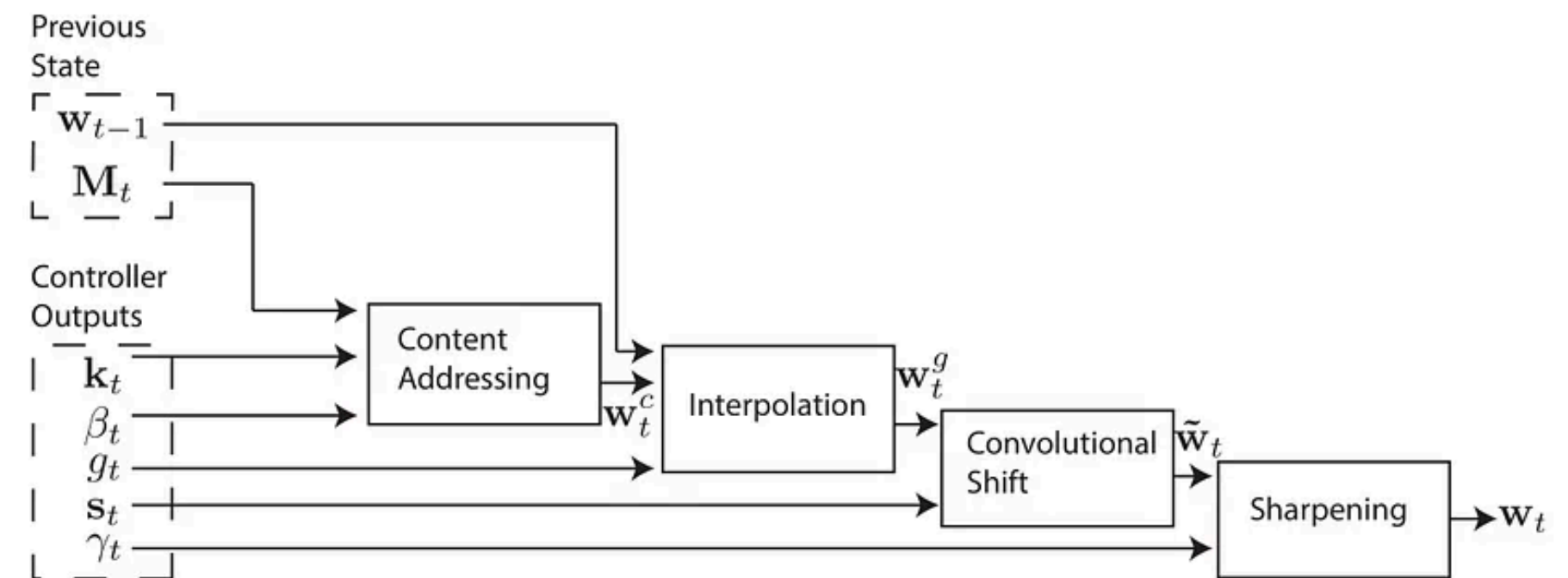
Tudo é treinado junto, com backpropagation.



ENDEREÇAMENTO DA MEMÓRIA

Como a NTM escolhe onde acessar na memória?

- Vetor de atenção $w^t \in \mathbb{R}^N$, com $\sum w_i = 1$
- Dois modos de endereçamento:
 - Content-based
 - Compara uma chave com as células da memória
 - Usa similaridade do cosseno
 - Resultado passa por softmax
 - Location-based
 - Interpola com o vetor anterior
 - Aplica deslocamento circular (shift)
 - Finaliza com sharpening
- Tudo isso é diferenciável → permite backpropagation fim a fim



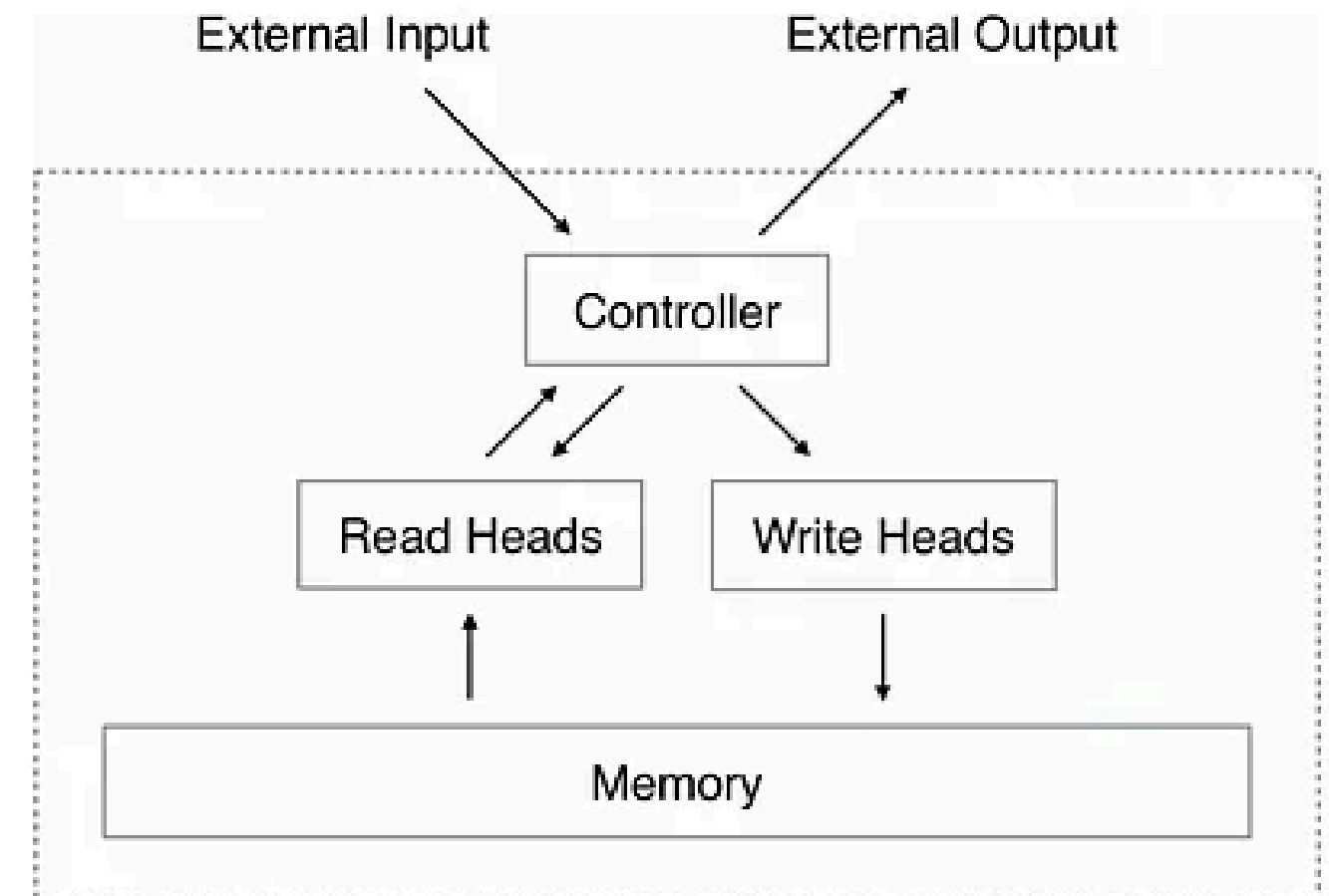
OPERAÇÕES DE LEITURA E ESCRITA

Leitura:

- A NTM gera um vetor de leitura:
 - $\mathbf{r} = \mathbf{w}^t \cdot \mathbf{M}$
- Combinação ponderada das posições da memória.
- É enviado para o controlador como entrada.

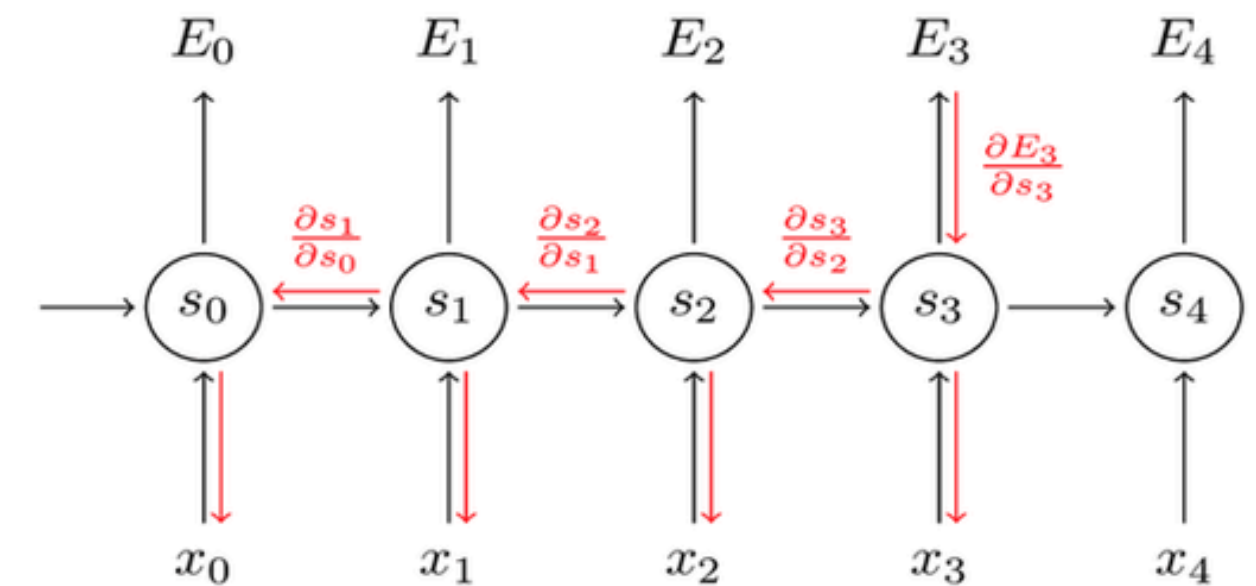
Escrita (em duas etapas):

- Erase:
 - $\mathbf{M} \leftarrow \mathbf{M} \odot (1 - \mathbf{w} \cdot \mathbf{e}^t)$
- Add:
 - $\mathbf{M} \leftarrow \mathbf{M} + \mathbf{w} \cdot \mathbf{a}^t$
- O controlador produz os vetores erase e add a cada passo.



Como a NTM é treinada?

- Função de perda externa:
 - Usa MSE, cross-entropy, etc.
 - A NTM aprende com base nas saídas esperadas
- Backpropagation Through Time (BPTT):
 - O erro se propaga por:
 - Controller
 - Heads de leitura e escrita
 - Memória externa
 - Tudo é diferenciável → permite treinamento fim a fim

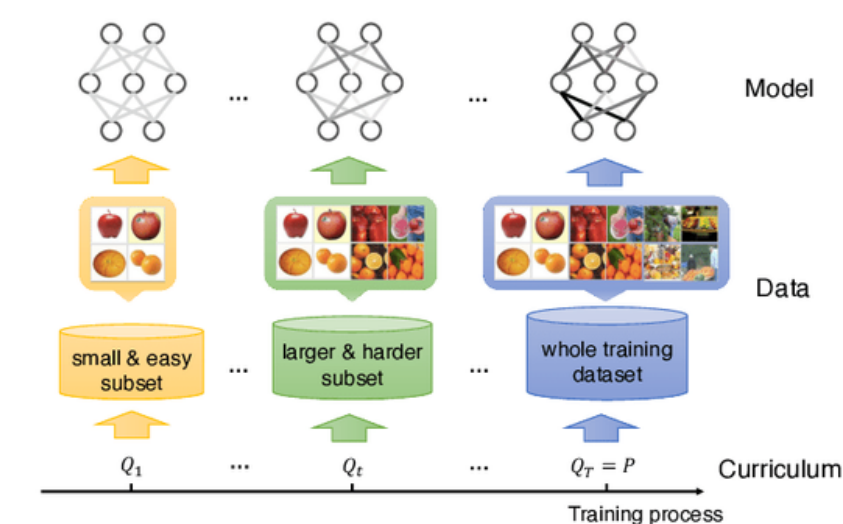
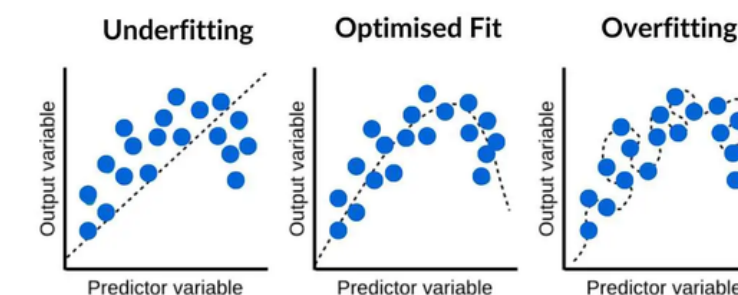
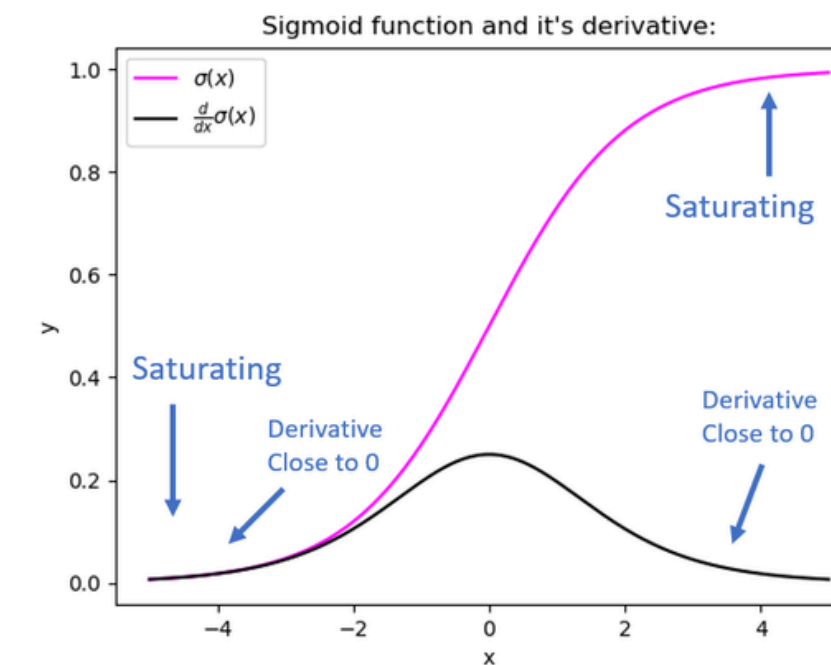


Backpropagation Through Time

DESAFIOS NO TREINAMENTO

Desafios no treinamento da NTM

- Problemas enfrentados:
 - Gradientes muito longos → instabilidade
 - Dificuldade para convergir em tarefas complexas
- Estratégias para estabilizar:
 - Gradient clipping
 - Curriculum learning (começar com tarefas fáceis)
 - Regularização (ex: weight decay)



TAREFAS-BENCHMARK ORIGINAIS

Tarefa	Meta	Quem falha / quem aprende
Copy	Reproduzir sequência qualquer	LSTM falha em seq. longas ↔ NTM generaliza
Repeat-Copy	Copiar N vezes	NTM converge
Associative Recall	Recuperar valor dado chave	NTM aprende
Priority Sort	Ordenar lista	NTM aprende

COMPARAÇÕES & LIMITAÇÕES

Vantagens:

- Melhor que LSTM em tarefas como copiar e ordenar
- Generaliza para sequências mais longas do que as vistas no treinamento
- Capaz de manipular dados como um array, usando memória endereçável

Limitações:

- Custo computacional alto:
 - Memória de dimensão $N \times M$
- Treinamento instável:
 - Difícil de convergir sem técnicas de ajuste (clipping, curriculum, etc.)
- Memória “soft”:
 - Escrita suave (não binária) → introduz ruído

FUTURO & DESAFIOS ABERTOS

- **Eficiência de memória**
 - **Acesso esparsos e compressão inteligente**
- **Estabilidade em sequências longas**
 - **Mesmo com memória externa, os gradientes se degradam**
- **Combinando com Reinforcement Learning**
 - **NTMs como agentes que planejam e interagem**
- **Escalabilidade prática**
 - **Latência, throughput e integração ainda são barreiras**

O NOSSO PROJETO

O que foi feito:

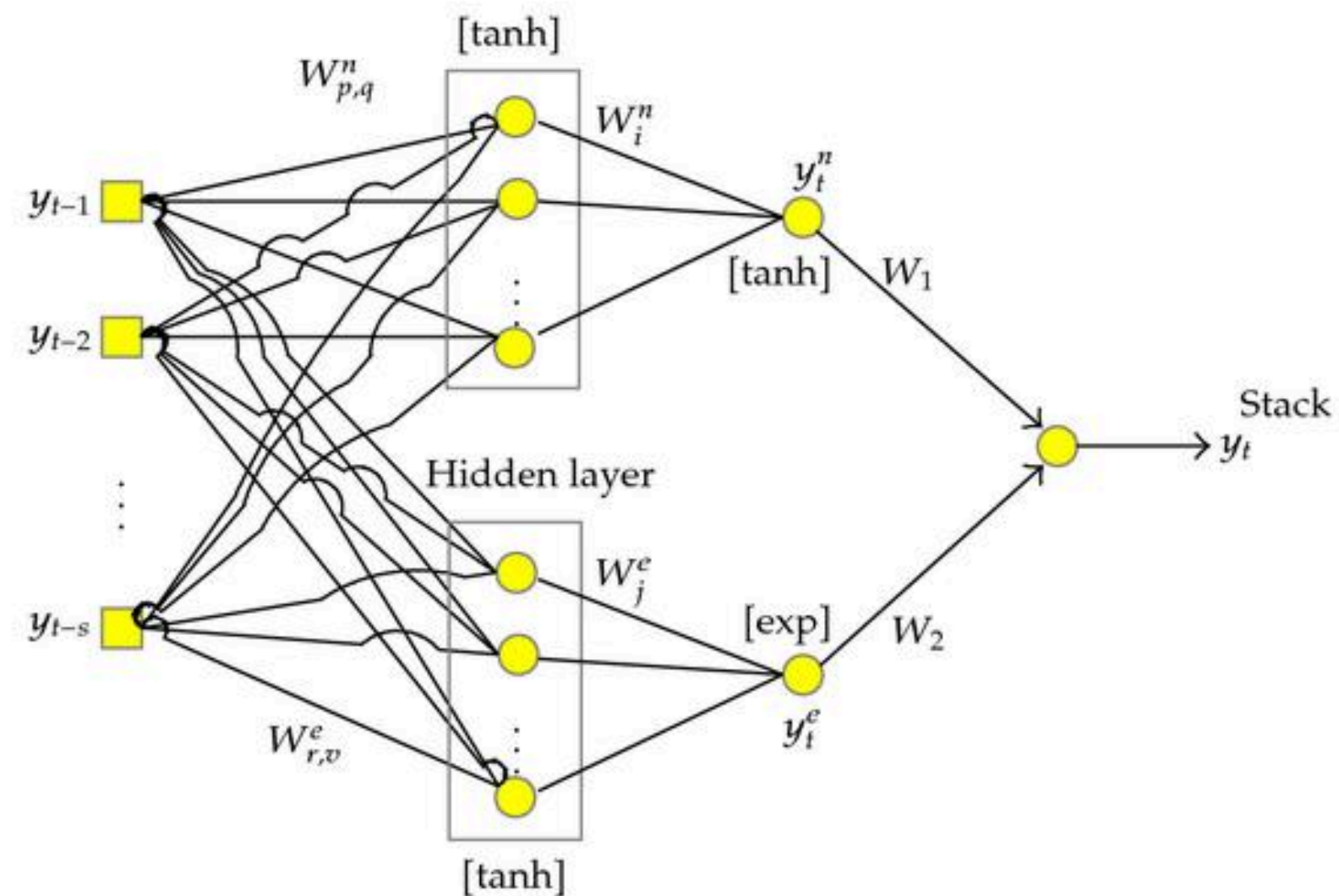
- Utilizamos uma Neural Turing Machine para simular um Pushdown Automaton (PDA)
- A memória da NTM foi tratada como uma pilha LIFO

Objetivo:

- Ensinar a NTM a reconhecer parênteses balanceados
 - $((()))$, $((()()))$, mas não $((() ou (()(($

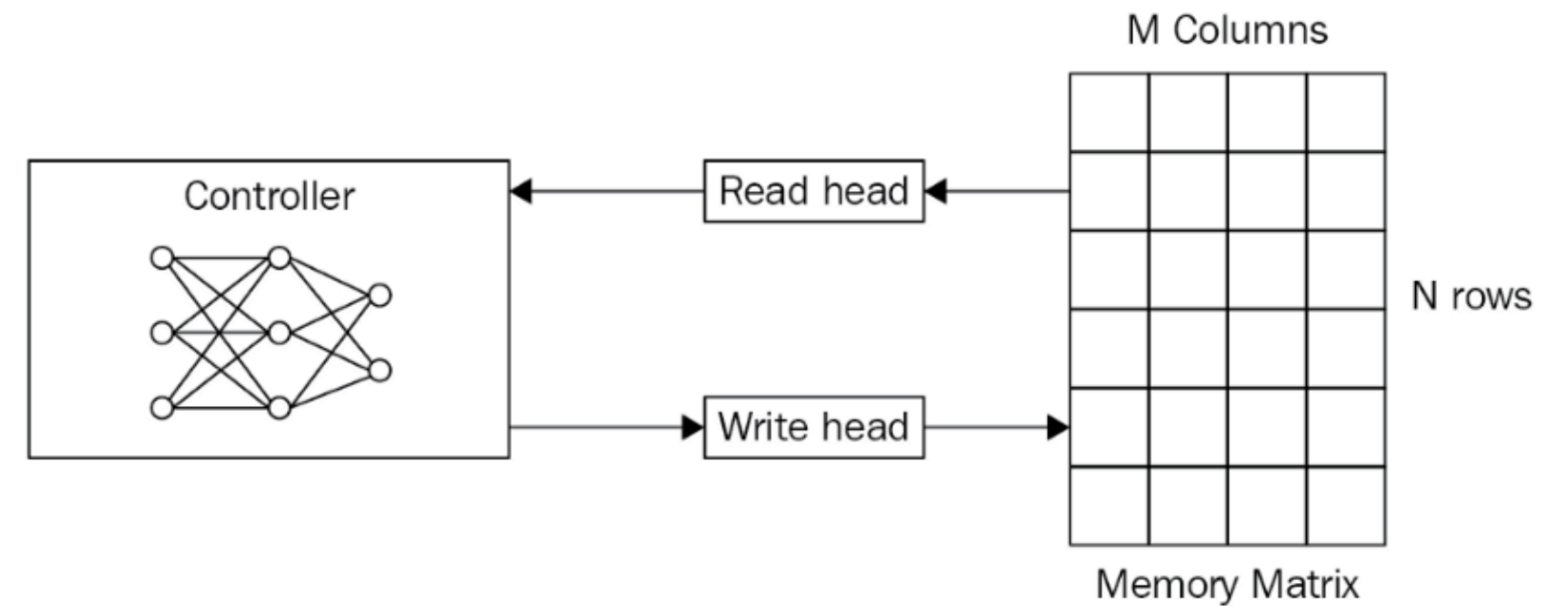
Por que isso importa?

- Mostra que a NTM aprende estruturas algorítmicas
- Abre caminho para aplicações em análise de código, interpretação de linguagens e IA simbólica



O NOSSO PROJETO

- **Arquitetura**
 - **Controller (RNN)**
 - **Memória externa diferenciável**
 - **Cabeças de leitura/escrita com vetores de atenção**
- **Aprendizado**
 - **Treinamento end-to-end via gradiente**
 - **Aprende a organizar dados sequenciais (pilha, lista)**
- **Capacidade**
 - **Simula pilha LIFO**
 - **Resolve tarefas de linguagens livres de contexto**



NTM/MEMORY.PY – MÓDULO DE MEMÓRIA

- **Propósito**
 - Implementa a memória externa endereçável da NTM
- **Classe principal: NTMMemory**
 - Matriz de memória de dimensão $N \times W$
 - Cada linha = uma posição de memória
- **Métodos principais:**
 - `reset()`: Inicializa a memória
 - `read(weights)`: Leitura com vetor de atenção
 - `write(weights, erase, add)`: Escrita com apagamento + adição
 - `address(key,  $\beta$ , g, s,  $\gamma$ , w_prev)`: Endereçamento por conteúdo + localização

A memória é o coração da NTM, permitindo armazenar e recuperar informações de forma diferenciável e controlada pela rede.

NTM/CONTROLLER.PY – MÓDULO CONTROLADOR

- **Propósito**
 - Atuar como o “cérebro” da NTM
 - Decide o que escrever, ler e produzir
- **Classe principal: NTMController**
 - Rede feed-forward (Linear + ReLU)
 - Recebe:
 - Entrada externa
 - Vetores lidos da memória
 - Produz:
 - Saída final
 - Parâmetros para as cabeças de leitura e escrita
- **Métodos principais**
 - `forward()` → Processa entrada e gera comandos
 - `_get_read_params()` → Chave, força, interpolação, shift
 - `_get_write_params()` → Vetores de apagamento e adição

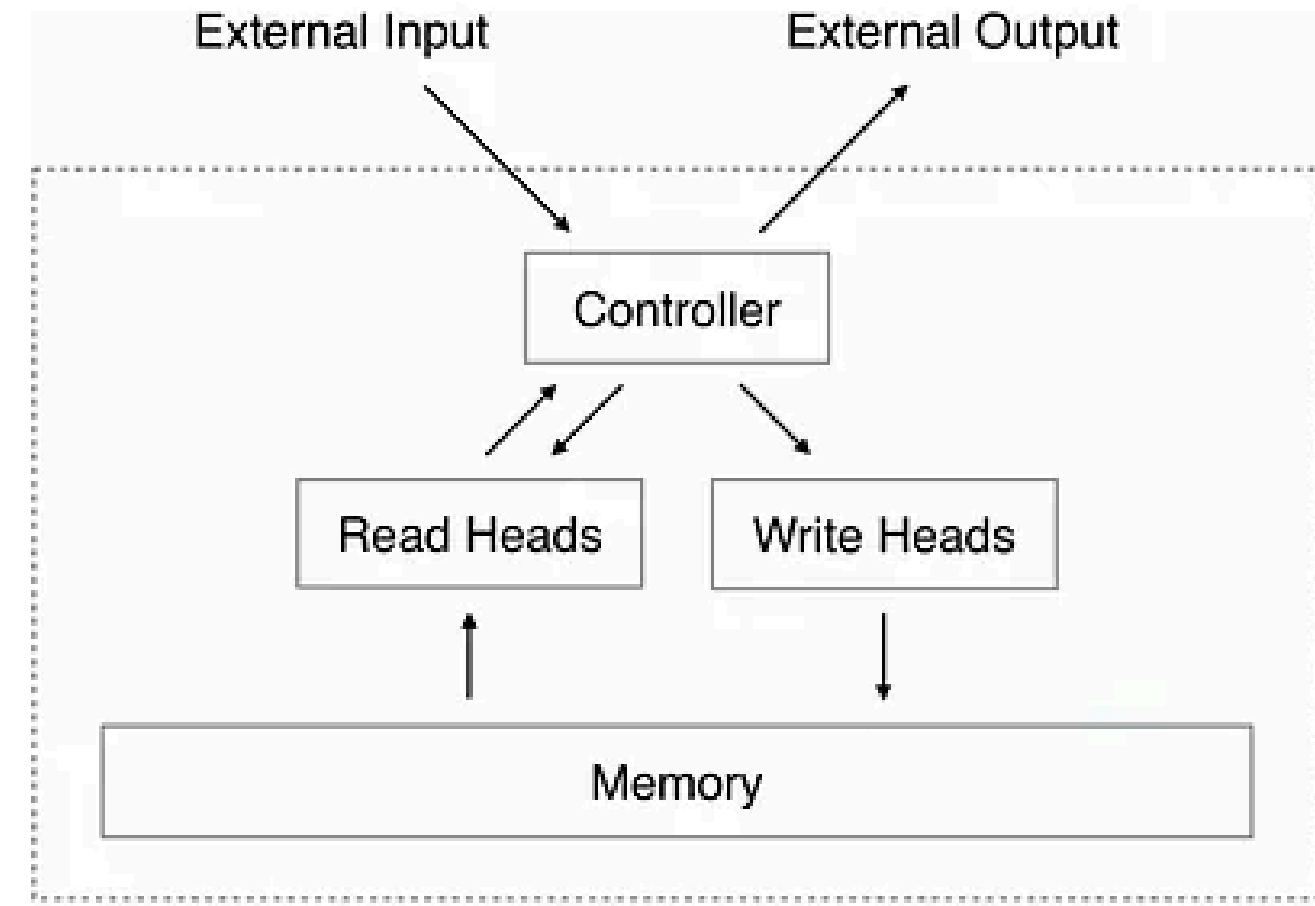
- **Propósito**
 - Atua como a “cola” da arquitetura
 - Orquestra o controlador, memória e cabeças
- **Classe: NeuralTuringMachine**
 - Integra:
 - NTMController
 - NTMMemory
- **Métodos principais**
 - `reset()` → Inicializa todos os estados
 - `forward()` → Processa um passo de tempo
 - `forward_sequence()` → Processa toda a sequência de entrada

Este módulo gerencia o fluxo total de dados, conectando entrada, controlador e memória de forma coordenada e diferenciável.

FLUXO DE EXECUÇÃO DA NTM

Etapas do Ciclo de Execução

1. **Entrada recebida**
2. **Controller processa entrada + vetores lidos**
3. **Gera parâmetros para leitura/escrita**
4. **Cabeças de escrita modificam a memória**
5. **Cabeças de leitura extraem nova informação**
6. **Controller gera saída com base no novo estado**



O NOSSO PROJETO

Reconhecimento de Parênteses Balanceados

- **Desafio:** verificar se uma sequência de (e) está balanceada
- **Ex:**
 - ✓ `((()))`, `((()()))`
 - ✗ `((()`, `((()((`

Abordagem adotada:

- Pushdown Automaton (PDA) clássico
- NTM emula a pilha → criando um Neural Pushdown Automaton (NTMPDA)
- Base: `examples/test_pushdown_automaton.py`

```
Estados      : q0, qaccept, qreject
Alfabeto     : (  )
Alf. pilha   : $ (          # $ = fundo
Início       : q0 (pilha: $)
Finais       : {qaccept}
```

```
(q0, '(', '$') → (q0, push '(')
(q0, '(', '(') → (q0, push '(')
(q0, ')', '(') → (q0, pop)
(q0, ε, '$') → (qaccept, $)      # cadeia vazia, pilha vazia
(q0, ε, '(') → (qreject, '(')    # sobrou '('
(q0, ')', '$') → (qreject, $)    # ')' sem par
```

Por que usar um NTMPDA?

- **Objetivo:**
 - Aprender comportamento de pilha, sem regras fixas

Como a NTM aprende?

- **Controlador neural**
 - Gera chaves, shifts, e sinais de controle
- **Memória externa**
 - $N \times M$ células acessadas com vetores de atenção
- **Cabeças de leitura/escrita**
 - Aprendem operações tipo push/pop

Arquitetura NTMPDA

```
class NTMPDA(nn.Module):  
    self.ntm = NeuralTuringMachine(  
        input_size =  $|\Sigma| + \epsilon$ ,  
        output_size = 1,          # logit  
        controller_size = 64,  
        memory_size = 32,  
        memory_locations = 64)  
    forward(x_batch) → ntm.forward_sequence()[-1]
```

O NOSSO PROJETO

- **Geração de Dados**
 - 50% balanceadas, 50% desequilibradas
 - 1.000 amostras de treino:
 - 500 balanceadas (label = 1)
 - 500 desequilibradas (label = 0)
 - 200 amostras de teste (100/100)
- **Pré-processamento**
 - One-hot por caractere
 - Padding até comprimento máximo (≤ 12)
- **Configuração de Treinamento**
 - Batch único (dataset inteiro de uma vez)
 - 100 épocas (num_epochs = 100)
 - Otimizador: Adam (lr = 1e-3)
 - Função de perda: BCEWithLogitsLoss → com pos_weight para balancear classes

DEMONSTRAÇÃO & FUNCIONAMENTO

- Demonstração Interativa
 - Usuário digita uma cadeia no terminal
 - NTMPDA retorna probabilidade da sequência ser balanceada
 - PDA clássico dá o veredito (aceita/rejeita)
- Como a NTM emula a pilha?
 - Push: controlador gera chave → desloca ponteiro → escreve vetor
 - Pop: ponteiro se move ao contrário → leitura ignora posição removida
 - Tudo é diferenciável → permite aprender push/pop via gradiente

- Conclusões
 - PDA tradicional serve como ground truth
 - NTMPDA aprende estrutura LIFO via treinamento
 - Aplicável a outras linguagens livres-de-contexto:
 - JSON
 - Expressões Regulares
 - Parsers simples

RESULTADOS DE TREINAMENTO DO NTMPDA

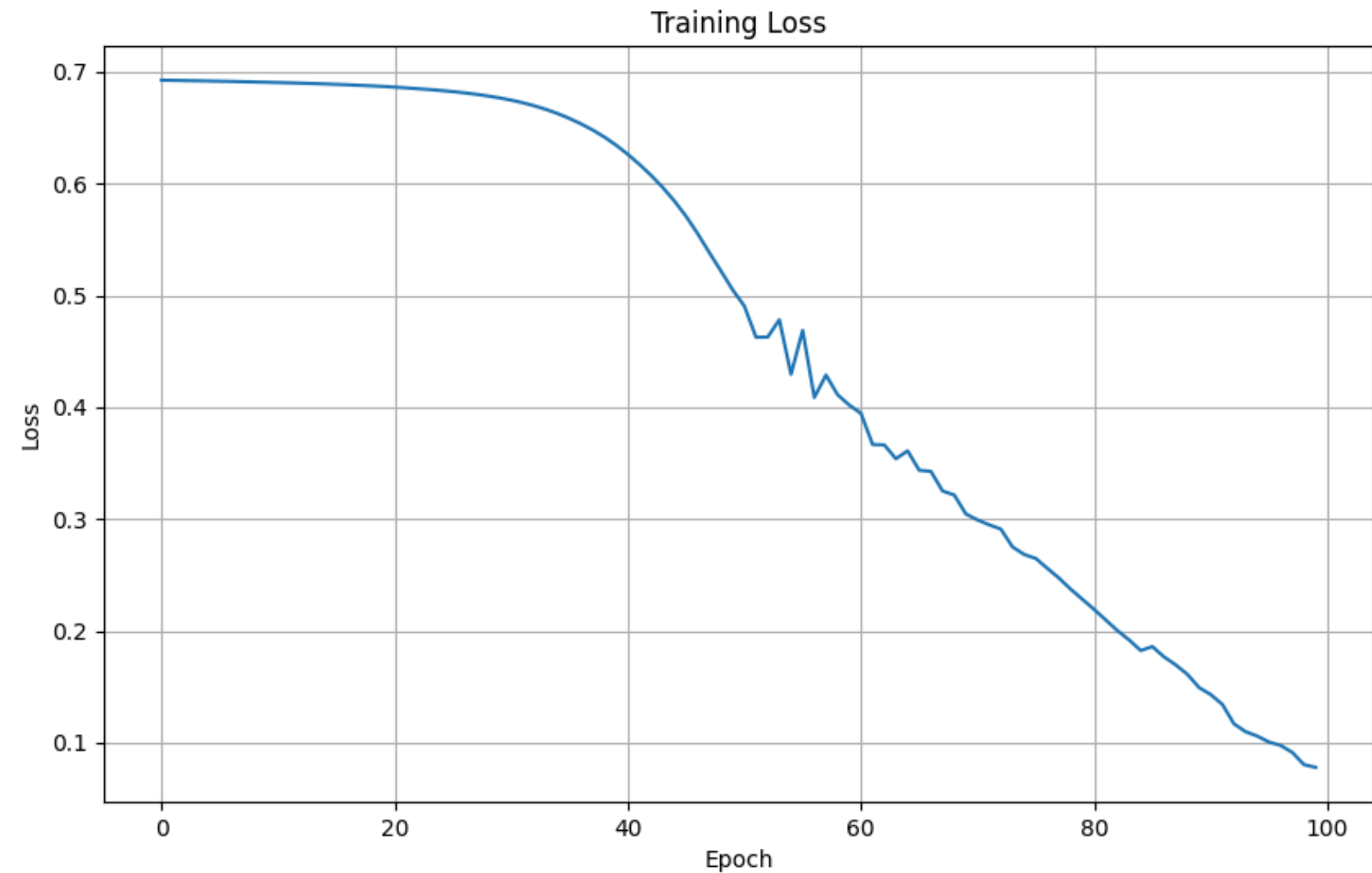
Testing pushdown automaton on examples:

String: '', Accepted: True
String: '()', Accepted: True
String: '(())', Accepted: True
String: '(()())', Accepted: True
String: '(((()))', Accepted: True
String: '(()())', Accepted: True
String: ')', Accepted: False
String: '(', Accepted: False
String: '())', Accepted: False
String: '(()', Accepted: False

Training neural pushdown automaton...

Simulating neural pushdown automaton training...

Epoch 10/100, Loss: 0.691240
Epoch 20/100, Loss: 0.688362
Epoch 30/100, Loss: 0.681198
Epoch 40/100, Loss: 0.645390
Epoch 50/100, Loss: 0.581479
Epoch 60/100, Loss: 0.499960
Epoch 70/100, Loss: 0.502756
Epoch 80/100, Loss: 0.416270
Epoch 90/100, Loss: 0.342596
Epoch 100/100, Loss: 0.231079
Final training loss: 0.231079
Training loss plot saved as 'pda_training_loss.png'



RESULTADOS DE TESTE E VALIDAÇÃO

Testing neural pushdown automaton on held-out data...

Confusion matrix (rows=Actual, cols=Pred):

	Pred 0	Pred 1
Actual 0	88	12
Actual 1	1	99

Accuracy : 0.94

Precision: 0.89

Recall : 0.99

F1 score : 0.94

```
String: '(()())', Pred: 1, Expected: 0
String: '((((()()))', Pred: 1, Expected: 1
String: '', Pred: 1, Expected: 1
String: '(()()())', Pred: 1, Expected: 1
String: '()', Pred: 0, Expected: 0
String: '))()())()', Pred: 1, Expected: 0
String: ')))()(', Pred: 0, Expected: 0
String: '))))()((((', Pred: 0, Expected: 0
String: '()', Pred: 1, Expected: 1
String: '()', Pred: 1, Expected: 1
String: '()())(', Pred: 0, Expected: 0
String: '()())(', Pred: 0, Expected: 0
String: '((((())', Pred: 1, Expected: 1
String: '((((()()))', Pred: 1, Expected: 1
String: ')))((', Pred: 0, Expected: 0
String: ')(((', Pred: 0, Expected: 0
String: '>()()(', Pred: 0, Expected: 0
String: '()', Pred: 1, Expected: 1
String: '(()()())(((', Pred: 0, Expected: 0
String: '((((()()))', Pred: 1, Expected: 1
```

Interactive testing. Enter strings of parentheses to test (empty line to quit).

Input: ()()

Neural PDA probability: 0.91 -> ACCEPT

Classical PDA verdict: ACCEPT

Input: ((

Neural PDA probability: 0.00 -> REJECT

Classical PDA verdict: REJECT

Input:)(

Neural PDA probability: 0.07 -> REJECT

Classical PDA verdict: REJECT

Input: ()()()

Neural PDA probability: 0.94 -> ACCEPT

Classical PDA verdict: ACCEPT

Input: (((()))

Neural PDA probability: 0.97 -> ACCEPT

Classical PDA verdict: ACCEPT

EXEMPLO

The image shows a VS Code editor window with a Python file named `test_pushdown_automaton.py` open. The file is located in the `examples` directory of a project named `LFA_P3`. The code defines a pushdown automaton for recognizing balanced parentheses and tests it on various strings.

```
6:
7: import torch
8: import torch.nn as nn
9: import torch.optim as optim
10: import numpy as np
11: import matplotlib.pyplot as plt
12:
13: from automata.pushdown_automaton import PushdownAutomaton, NeuralPushdownAutomaton
14: from ntm.ntm import NeuralTuringMachine
15:
16: def create_balanced_pda():
17:     """
18:     Create a pushdown automaton that recognizes balanced parentheses.
19:     """
20:     states = {'q0', 'q1', 'qaccept', 'qreject'}
21:     input_alphabet = {'(', ')'}
22:     stack_alphabet = {'$', '('} # $ is the bottom of stack marker
23:     transitions = {
24:         ('q0', '(', '$'): ('q0', ['$', '(']), # Push '(' onto stack
25:         ('q0', '(', '('): ('q0', ['$', '(']), # Push '(' onto stack
26:         ('q0', ')', '('): ('q0', []), # Pop '(' from stack
27:         ('q0', '', '$'): ('qaccept', ['$']), # Empty string is accepted
28:         ('q0', '', '('): ('qreject', ['(']), # Unmatched '(' at the end
29:         ('q0', ')', '$'): ('qreject', ['$']), # Unmatched ')' at the beginning
30:     }
```

The terminal output shows the execution of the script:

```
caio@caio-550XDA:~/Área de trabalho/lfa_p3$ "/home/caio/Área de trabalho/lfa_p3/.venv/bin/python" "/home/caio/Área de trabalho/lfa_p3/examples/test_pushdown_automaton.py"
Creating a pushdown automaton that recognizes balanced parentheses...

Testing pushdown automaton on examples:
String: '', Accepted: True
String: '()', Accepted: True
String: '()', Accepted: True
String: '()', Accepted: True
String: '()', Accepted: True
String: '()', Accepted: True
String: '()', Accepted: True
String: '()', Accepted: False
String: '()', Accepted: False
String: '()', Accepted: False
String: '()', Accepted: False

Training neural pushdown automaton...

Simulating neural pushdown automaton training...
```

NTM APRENDENDO LINGUAGENS REGULARES (DFA)

Objetivo

- Ensinar uma Neural Turing Machine (NTM) a simular o comportamento de um DFA simples

O que o DFA faz

- Aceita apenas cadeias que terminam em "01"
- Usa transições determinísticas com 2 ou 3 estados

Motivação

- Mostrar que a NTM pode aprender linguagens regulares
- Reforçar que a NTM pode simular autômatos clássicos via aprendizado

Geração do dataset

- 500 positivos: cadeias que terminam com "01"
- 500 negativos: sorteio com viés para cadeias curtas
- Tamanho máximo: 12 símbolos
- Dados balanceados

Modelo NTMDFA

- Subclasse com um objeto `NeuralTuringMachine`
- Processa sequência inteira
- Última saída passa por sigmoid para gerar a probabilidade de aceitação da cadeia

Treinamento

- Junta positivos e negativos, embaralha e cria DataLoader (batch = 64)
- Perda: BCELoss
- Otimizador: Adam (lr = $1e-3$), 100 épocas
- Aplica gradient clipping (norma ≤ 5)
- Salva a loss a cada época (para plot posterior)

Teste

- Usa 10 exemplos manuais fixos
- Predição com threshold de 0.6
- Exibe:
 - Matriz de confusão
 - Accuracy, Precision, Recall, F1 Score

COMO A NTM APRENDE O DFA

- A NTM aprende a reconstruir o estado do DFA internamente.
- Armazena os últimos dois símbolos da entrada na memória externa.
- Ajusta suas cabeças de leitura/escrita para atualizar esse "registro" a cada passo.
- O treinamento supervisionado minimiza a BCE entre saída e rótulo (0 ou 1).
- Após 100 épocas, o loss $\approx 1e-5$ e a NTM atinge 100% de acurácia no teste reduzido.
- Na inferência, usa-se threshold 0.60 para aceitar ou rejeitar.

TESTES INTERATIVOS COM CADEIAS DE 0/1

Modo interativo via terminal

- Usuário digita cadeias de 0s e 1s
- NTM responde com:
 - Probabilidade de aceitação
 - Veredito final com threshold ($\geq 0.6 \rightarrow$ aceita)
 - Veredito do DFA clássico (comparativo)

NTM APRENDENDO DFA QUE RECONHECE '01'

```
Creating DFA that recognizes strings ending with '01'...  
Generating examples...
```

```
Positive examples (accepted by DFA):
```

```
1101  
0110010101  
001  
1010001001  
110001  
...
```

```
Negative examples (rejected by DFA):
```

```
01000000  
1  
10  
0  
11110100  
...
```

```
Training Neural Turing Machine-based automaton...
```

```
Epoch 10/100, Loss: 0.064078
```

```
Epoch 20/100, Loss: 0.002124
```

```
Epoch 30/100, Loss: 0.000390
```

```
Epoch 40/100, Loss: 0.000164
```

```
Epoch 50/100, Loss: 0.000088
```

```
Epoch 60/100, Loss: 0.000054
```

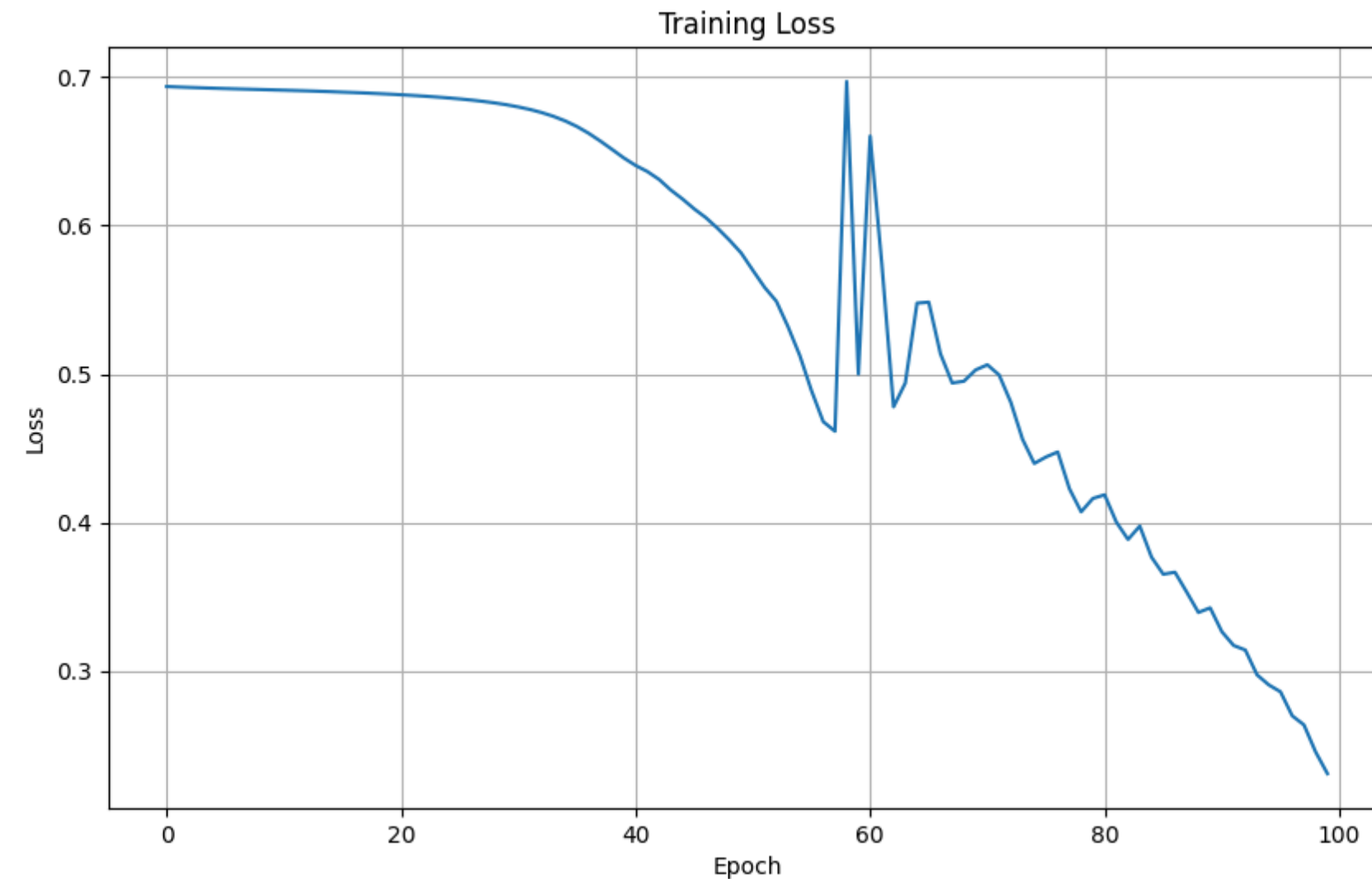
```
Epoch 70/100, Loss: 0.000035
```

```
Epoch 80/100, Loss: 1.893657
```

```
Epoch 90/100, Loss: 0.000533
```

```
Epoch 100/100, Loss: 0.000232
```

```
Training loss plot saved as 'training_loss.png'
```



RESULTADOS FINAIS – NTM SIMULANDO DFA

Testing Neural Turing Machine-based automaton...

Example: 01, Prediction: 1, Expected: 1

Example: 101, Prediction: 1, Expected: 1

Example: 0101, Prediction: 1, Expected: 1

Example: 1, Prediction: 0, Expected: 0

Example: 10, Prediction: 0, Expected: 0

Example: 011, Prediction: 0, Expected: 0

Example: 0011, Prediction: 0, Expected: 0

Example: 00101, Prediction: 1, Expected: 1

Example: , Prediction: 0, Expected: 0

Example: 0, Prediction: 0, Expected: 0

Confusion matrix (rows=Actual, cols=Pred):

	Pred 0	Pred 1
Actual 0	6	0
Actual 1	0	4

Accuracy : 1.00

Precision: 1.00

Recall : 1.00

F1 score : 1.00

Neural Turing Machine DFA accuracy: 1.00

Interactive testing. Enter strings of 0s and 1s to test (empty line to quit).

Input: 01

Neural TM probability: 0.84 -> ACCEPT

Classical DFA verdict: ACCEPT

Expected pattern: String should end with '01' to be accepted

Input: 11

Neural TM probability: 0.00 -> REJECT

Classical DFA verdict: REJECT

Expected pattern: String should end with '01' to be accepted

Input: 00

Neural TM probability: 0.07 -> REJECT

Classical DFA verdict: REJECT

Expected pattern: String should end with '01' to be accepted

Input: 0101

Neural TM probability: 0.99 -> ACCEPT

Classical DFA verdict: ACCEPT

Expected pattern: String should end with '01' to be accepted

EXEMPLO

The image shows a Visual Studio Code editor window with a Python file named `automaton_simulation.py` open. The file is located in the `examples` directory of a project named `LFA_P3`. The code imports `torch`, `torch.nn`, `torch.optim`, `numpy`, `sys`, `os`, and `matplotlib.pyplot`. It also adds the parent directory to the path and imports `NeuralTuringMachine` from `ntm.ntm` and `FiniteAutomaton`, `NeuralFiniteAutomaton` from `automata.finite_automaton`. A function `create_dfa_example()` is defined, which creates a simple DFA that recognizes strings ending with '01'.

```
1 import torch
2 import torch.nn as nn
3 import torch.optim as optim
4 import numpy as np
5 import sys
6 import os
7 import matplotlib.pyplot as plt
8
9 # Add parent directory to path
10 sys.path.append(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
11
12 from ntm.ntm import NeuralTuringMachine
13 from automata.finite_automaton import FiniteAutomaton, NeuralFiniteAutomaton
14
15 def create_dfa_example():
16     """
17     Create a simple DFA that recognizes strings ending with '01'.
18
19     Returns:
20         FiniteAutomaton: A DFA that recognizes strings ending with '01'
21     """
```

The terminal output shows the execution of the script. It creates a DFA that recognizes strings ending with '01' and generates examples. The output lists positive examples (accepted by DFA) and negative examples (rejected by DFA).

```
caio@caio-550XDA:~/Área de trabalho/lfa_p3$ "/home/caio/Área de trabalho/lfa_p3/.venv/bin/python" "/home/caio/Área de trabalho/lfa_p3/examples/automaton_simulation.py"
Creating DFA that recognizes strings ending with '01'...
Generating examples...

Positive examples (accepted by DFA):
00000101
01
110010001
11011000001
10111001101
...

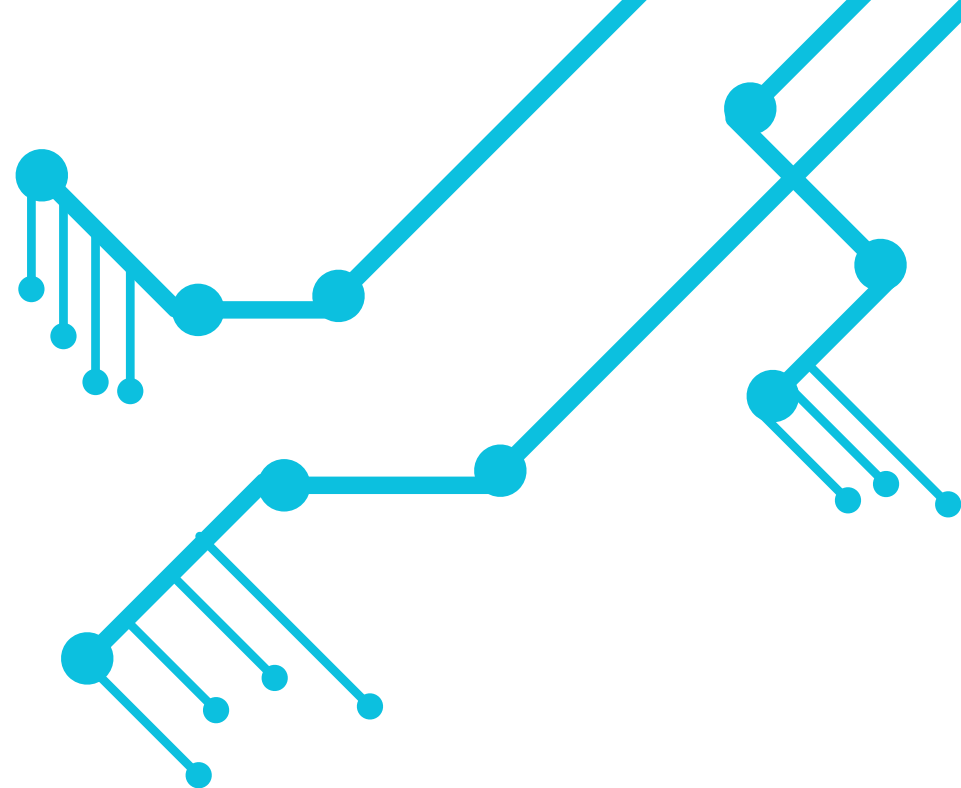
Negative examples (rejected by DFA):
011
11100010
10
0
11
...

Training Neural Turing Machine-based automaton...
```

OUTROS CASOS DE USO AVALIADOS

- Verificação de Palíndromos (NPDA)
- Expressões Matemáticas (PDA)
- Expressão Regular (AFD)

- **O que mostramos**
 - A NTM pode aprender operações de pilha (LIFO) via gradiente.
 - Foi capaz de simular autômatos clássicos (PDA e DFA).
 - Obteve alta acurácia e generalização em exemplos fora do treino.
- **Limitações**
 - Treinamento instável, exige cuidados (clipping, etc).
 - Alto custo computacional, pouco prático em larga escala.
- **Próximos passos**
 - Testar com palíndromos, expressões matemáticas e regex.
 - Explorar arquiteturas mais eficientes (p.ex., memórias esparsas).



REFERÊNCIAS

- Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation. Pearson Education.
- Jurafsky, D., & Martin, J. H. (2009). Speech and Language Processing. Prentice Hall.
- Cunha, C., & Cintra, L. (2013). Nova Gramática do Português Contemporâneo. Lexikon.
- GRAVES, Alex; WAYNE, Greg; DANIEHELKA, Ivo. Neural Turing Machines. arXiv preprint arXiv:1410.5401, 2014. Disponível em: [Hopcroft, J. E., Motwani, R., & Ullman, J. D. \(2006\). Introduction to Automata Theory, Languages, and Computation. Pearson Education..](#) Acesso em: 01 jul. 2025.



Obrigado pela atenção!

ALGUMA PERGUNTA?